Technical Report: Single-Store Walk-Forward Validation & LLM Forecasting Experiment

Date: April 6, 2026 **Prepared for:** Yahya Hammoudeh **Supervisor:** Mohammed Rida

---

Executive Summary

This report documents two experiments: (1) a walk-forward validation testing 20 retraining configurations for single-restaurant deployment, and (2) an evaluation of LLM-based demand forecasting as an alternative to traditional ML models. The key finding is that for per-store models retrained every 3 days, a **60-day sliding window with no exponential decay** produces the lowest business cost. Counterintuitively, decay weighting — which improved the global production model — **hurts** per-store models because the sliding window already handles recency. The LLM forecasting experiment shows that Haiku achieves MAE of 37-44 (vs ML's 8.5), making it 4-5x worse as a primary forecaster but potentially useful for cold-start scenarios.

Motivation

Restaurants change frequently: menus rotate, new promotions launch, foot traffic shifts seasonally. A model trained once on 6 months of data goes stale quickly. The production system needs a retraining strategy that balances freshness against stability.

The question: **What training window, recency weighting, and retraining frequency produce the best per-store predictions?**

Experiment 1: Walk-Forward Validation

Setup

Parameter	Value
Store	Zeynos (id=5995723) — high-volume shawarma restaurant
Items	Top 30 by volume (top: Hj Pita Shawarma, ~120 units/day)
Data	182 days, 5,460 rows (30 items x 182 days)
Model	LightGBM (200 trees, lr=0.05, 31 leaves, min_child=5)
Features	21 (day-of-week, lags 1/3/7/14d, rolling 3/7/14d, expanding mean, trend)
Retraining	Every 3 days, predict next 3 days
Min training	30 days before first prediction
Total retrains	51 over 153 test days

Parameter	Value
Evaluation	Shelf-life-aware business cost

Configurations Tested (20 total)

Crossed window sizes (14d, 21d, 30d, 45d, 60d, 90d, full) with decay half-lives (3d, 7d, 14d, none).

Results: Top 10 by Business Cost

Rank	Config	Cost (DKK)	MAE	Waste	Stockout
1	**60d window, no decay**	**1,574,987**	8.48	288,601	857,590
2	Full history, no decay	1,578,532	8.20	278,184	866,898
3	21d window, 7d decay	1,579,963	8.16	252,780	884,788
4	14d window, no decay	1,590,878	8.23	254,498	890,920
5	30d window, no decay	1,591,244	8.08	254,347	891,265
6	45d window, 7d decay	1,605,372	8.47	269,494	890,585
7	14d window, 7d decay	1,611,130	8.21	256,083	903,364
8	14d window, 3d decay	1,613,821	8.32	255,816	905,337
9	30d window, 14d decay	1,624,939	8.16	257,827	911,408
10	45d window, 14d decay	1,627,052	8.61	283,950	895,401

Bottom 5 (worst)

Rank	Config	Cost (DKK)
16	Full history, 14d decay	1,679,476
17	30d window, 3d decay	1,687,018
18	Full history, 7d decay	1,703,365
19	90d window, 7d decay	1,733,920
20	**Full history, 3d decay**	**1,795,315**

Key Finding: Decay Hurts Per-Store Models

The top 2 configs and 4 of the top 5 use **no decay**. The bottom 5 all use aggressive decay (3d or 7d). This is the opposite of the production-scale global model, where decay=6d won.

Why this happens:

Approach	Global model (494 stores)	Per-store model (1 store)
Training data	2.26M rows, full history	~5,400 rows, sliding window
Recency mechanism	Exponential decay on weights	Window drops old data directly
Retraining	Once (offline)	Every 3 days
Decay effect	Useful — down-weights stale cross-store patterns	Harmful — double-discounts recent data that's already emphasized by the window

When retraining every 3 days on a 60-day window, old data is already excluded. Adding exponential decay on top makes the model forget weekly patterns too aggressively. A Friday pattern seen 4 weeks ago is still informative — decay=3d would nearly zero it out.

The sliding window IS the recency mechanism. Decay is redundant.

Window Size Analysis

Window	Cost (no decay)	Why
14 days	1,590,878	2 weekly cycles — underfits day-of-week patterns
30 days	1,591,244	4 cycles — good but slight instability
60 days	**1,574,987**	8 cycles — best: enough diversity, no stale noise
90 days	(not tested w/o decay)	Stale patterns from 3 months ago drag quality
Full	1,578,532	Competitive (#2), but vulnerable to distribution shifts

Sweet spot: **30-60 days**. Below 30, the model lacks enough weekly repetitions to learn stable day-of-week effects. Above 60, old patterns that no longer reflect current behavior degrade predictions.

Buffer Sweep (60d window, no decay)

Buffer	Cost (DKK)	Waste	Stockout	MAE
1.00	1,980,590	200,329	1,186,841	7.63
1.10	1,691,153	257,540	955,742	8.13
1.20	1,474,646	320,979	769,111	8.89
1.30	1,320,235	389,451	620,523	9.88
1.40	**1,223,400**	462,535	507,244	11.07

Higher buffers keep winning because the stockout penalty (1.5x) combined with shelf-life-aware waste fractions (most items have low effective waste) means over-ordering is cheap.

Experiment 2: LLM Forecasting

Can a language model predict demand from raw sales numbers?

Tested Haiku (Claude's fast model) as a demand forecaster for Zeynos's top item (Hj Pita Shawarma, ~120 units/day).

Test A: One-Shot (126 days history -> predict 55 days)

Haiku received 126 days of daily sales with dates and day-of-week labels. It predicted all 55 test days in a single response.

Metric	Value
MAE	37.3
Avg prediction	95.7 (actual avg: 116.5)
Within 20 units	33% of days
Business cost	113,926 DKK

Failure mode: Haiku saw a downward trend (Oct avg 150 -> Jan-Feb avg 100) and extrapolated ~95 for all 55 days. Actual demand rebounded to ~117. The model cannot adapt without feedback.

Test B: Rolling (14-day window -> predict 3 days, slide 3)

19 prediction rounds. Each round: Haiku sees the most recent 14 actual days, predicts the next 3. Windows pre-computed with actual data (not predicted) for the shift.

Metric	Rolling (14d->3d)	One-Shot (126d->55d)
MAE	43.5	37.3
Business cost	**103,342 DKK**	113,926 DKK
Avg prediction	112.2	95.7

Rolling has worse MAE but better business cost — it tracks the actual demand level better (avg 112 vs 96, actual 117), producing fewer stockouts.

LLM vs ML Comparison

Forecaster	MAE	Business Cost
LightGBM walk-forward (60d, retrain/3d)	**~8.5**	**~28,000 DKK** (estimated for single item)
Haiku rolling (14d window)	43.5	103,342 DKK
Haiku one-shot (full history)	37.3	113,926 DKK

The ML model is **4-5x better** on MAE and **~4x better** on business cost. An LLM with 14 raw numbers is doing weighted averages from pattern recognition — it has no lag features, no rolling statistics, no learned non-linear relationships.

Where LLM Forecasting Could Be Useful

Despite poor primary performance, LLM forecasting has potential as:

1. **Cold start** — new store with 7-14 days of data, no model trained yet. LLM gives a reasonable first approximation from raw sales.
2. **Anomaly detection** — if ML predicts 50 and LLM predicts 150 from the same history, something unusual may be happening.
3. **Fine-tuned specialist** — a small model (Gemma 4, Phi-3) fine-tuned specifically on demand time series could learn the forecasting task structurally, not just from text pattern-matching.

Recommended Production Config (Per-Store)

Parameter	Value	Rationale
Model	LightGBM, 200 trees, lr=0.05, 31 leaves	Fast, handles small data well
Window	60-day sliding	8 weekly cycles, no stale noise
Decay	None	Window handles recency; decay is redundant
Retrain	Every 3 days	Adapts to menu/behavior changes
Buffer	1.30-1.40	Higher is better with shelf-aware waste
Features	21 (lightweight)	Lags, rolling means, day-of-week, trend
Retrain time	~0.5s per store	494 stores x 0.5s = ~4 min total
Fallback	Global model	For stores with <30 days history

Global vs Per-Store Trade-Off

Aspect	Global Model	Per-Store Model
Training data	2.26M rows (all stores)	~5,400 rows (one store)

Aspect	Global Model	Per-Store Model
Strengths	Cross-store pattern sharing, robust for thin data	Adapts to local changes, no stale cross-store noise
Weaknesses	Slow to adapt, one-size-fits-all	Needs 30+ days history, noisier
Best for	New stores, stores with sparse data	Established stores, frequent menu changes
Recency	Needs decay (6d)	Needs window (60d), no decay

Production system should use both: global model as fallback for new stores, per-store model where history permits (>30 days).

Files

File	Description
`notebooks/single_store_experiment.py`	Walk-forward validation script
`notebooks/results/single_store_regression.json`	20-config comparison results
`data/raw/rolling_windows.json`	LLM prediction windows
`data/raw/rolling_predictions.jsonl`	Haiku rolling predictions

Report generated: April 6, 2026